



**CHANDIGARH
UNIVERSITY**

Discover. Learn. Empower.

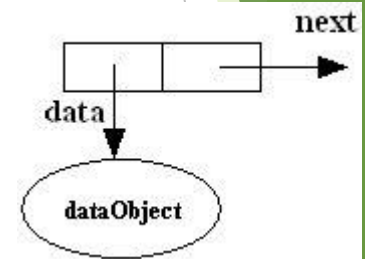
Topic 2.1

Linear Data Structures

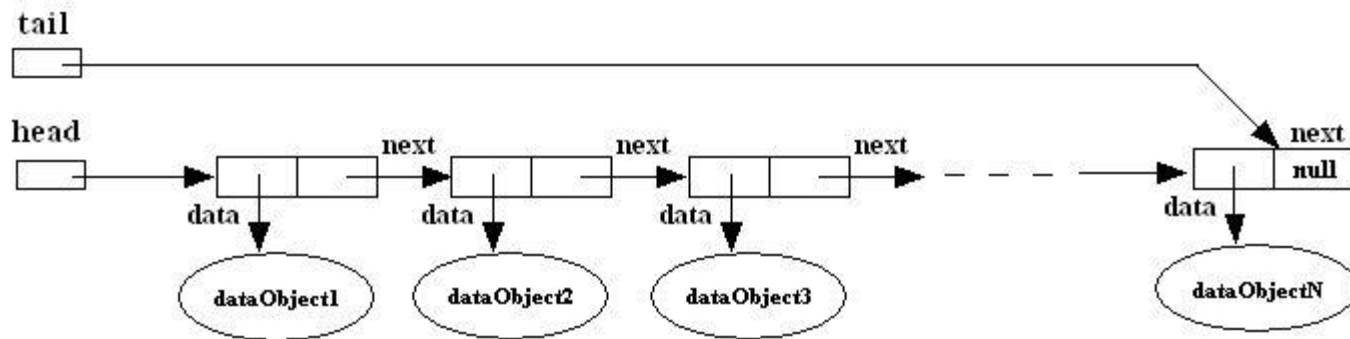
What is a Singly-linked list?

- ▶ A singly linked list is a dynamic data structure consisting of a sequence of nodes, forming a linear ordering.
- ▶ Each node stores:
 - ▶ Element (data object)
 - ▶ Reference (i.e., address) to the next node

Node:



Singly-linked list:



Linear Data Structures

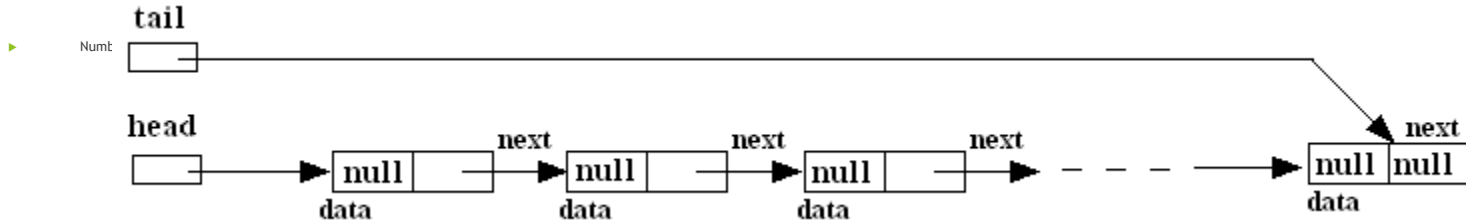
- A linear data structure is one where data items are arranged in a linear fashion. Each member is attached to its neighboring elements. The structure permits single-level data storage because the data elements are stored in a linear fashion.
- The data can be traversed in one run. A linear data structure does not maximize memory.
- A linear data structure is one where data items are ordered either sequentially or in a linear fashion, and each member is attached to its neighbouring elements.
- The linear data structure's elements can all be accessed in one run. There are four primary types of linear data structures: Queue, Linked Lists, Stack, and Arrays.

Singly-linked lists vs. 1D-arrays

ID-array	Singly-linked list
Fixed size: Resizing is expensive	Dynamic size
Insertions and Deletions are inefficient: Elements are usually shifted	Insertions and Deletions are efficient: No shifting
Random access i.e., efficient indexing	No random access → Not suitable for operations requiring accessing elements by index such as sorting
No memory waste if the array is full or almost full; otherwise may result in much memory waste.	Extra storage needed for references; however uses exactly as much memory as it needs
Sequential access is faster because of greater locality of references [Reason: Elements in contiguous memory locations]	Sequential access is slow because of low locality of references [Reason: Elements not in contiguous memory locations]

Representation: Space Analysis

- Assume:
 - There are n nodes in the list
 - All data references are null



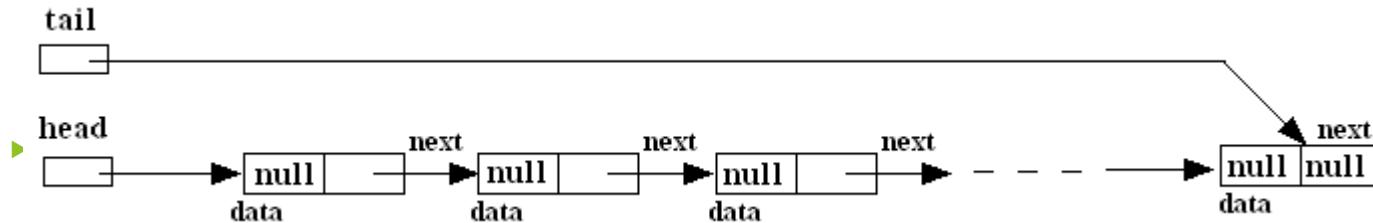
Reference	Total	Required space
head	1	SizeOfSinglyLinkedListElementReference
tail	1	SizeOfSinglyLinkedListElementReference
next	n	$n \times \text{SizeOfSinglyLinkedListElementReference}$
data	n	$n \times \text{SizeOfObjectReference}$

Total space = $(n + 2) \times \text{SizeOfSinglyLinkedListElementReference}$
+ $n \times \text{SizeOfObjectReference}$

Hence space complexity is $O(n)$

Representation: Space Analysis

- ▶ Assume:
 - ▶ There are n nodes in the list
 - ▶ All data references are null



Reference	Total	Required space
head	1	SizeOfSinglyLinkedListElementReference
tail	1	SizeOfSinglyLinkedListElementReference
next	n	$n \times \text{SizeOfSinglyLinkedListElementReference}$
data	n	$n \times \text{SizeOfObjectReference}$

Total space = $(n + 2) \times \text{SizeOfSinglyLinkedListElementReference}$
+ $n \times \text{SizeOfObjectReference}$

Hence space complexity is $O(n)$

Traverse a Linked List

- ▶ Displaying the contents of a linked list is very simple. We keep moving the temp node to the next one and display its contents.
- ▶ When temp is NULL, we know that we have reached the end of the linked list so we get out of the while loop.
- ▶ `struct node *temp = head;`

```
printf("\n\nList elements are - \n");
```

```
while(temp != NULL) {
```

```
printf("%d --->",temp->data); temp = temp->next;
```

```
}
```

Insert at the beginning

- ▶ Allocate memory for new node
- ▶ Store data
- ▶ Change next of new node to point to head
- ▶ Change head to point to recently created node

Insert at the End

Allocate memory for new node

Store data

Traverse to last node

Change next of last node to recently created node

```
struct node *newNode;  
newNode = malloc(sizeof(struct node));  
newNode->data = 4;  
newNode->next = NULL;  
  
struct node *temp = head;  
while(temp->next != NULL){  
    temp = temp->next;  
}  
  
temp->next = newNode;
```

Insert at the Middle

- ▶ Allocate memory and store data for new node
- ▶ Traverse to node just before the required position of new node
- ▶ Change next pointers to include new node in between

LIST-PREPEND(L, x)

```
1   $x.next = L.head$   
2   $x.prev = NIL$   
3  if  $L.head \neq NIL$   
4       $L.head.prev = x$   
5   $L.head = x$ 
```

LIST-INSERT(x, y)

```
1   $x.next = y.next$   
2   $x.prev = y$   
3  if  $y.next \neq NIL$   
4       $y.next.prev = x$   
5   $y.next = x$ 
```

Delete from beginning

- ▶ Point head to the second node
- ▶ `head = head->next;`

Delete from end

- ▶ Traverse to second last element
- ▶ Change its next pointer to null
- ▶ `struct node* temp = head;`
- ▶ `while(temp->next->next!=NULL)`
- ▶ `{`
- ▶ `temp = temp->next;`
- ▶ `}`
- ▶ `temp->next = NULL;`

Delete from middle

- ▶ Traverse to element before the element to be deleted
Change next pointers to exclude the node from the chain
for(int i=2; i< position; i++)

```
{  
    if(temp->next!=NULL) {  
temp = temp->next;  
    }  
}  
  
temp->next = temp->next->next;
```

LIST-DELETE(L, x)

```
1  if  $x.prev \neq \text{NIL}$   
2       $x.prev.next = x.next$   
3  else  $L.head = x.next$   
4  if  $x.next \neq \text{NIL}$   
5       $x.next.prev = x.prev$ 
```

Search an Element on a Linked List

- ▶ You can search an element on a linked list using a loop using the following steps. We are finding item on a linked list.
Make head as the current node.
Run a loop until the current node is NULL because the last element points to NULL.
In each iteration, check if the key of the node is equal to item. If it the key matches the item, return true otherwise return false.

LIST-SEARCH(L, k)

```
1   $x = L.head$   
2  while  $x \neq \text{NIL}$  and  $x.key \neq k$   
3       $x = x.next$   
4  return  $x$ 
```

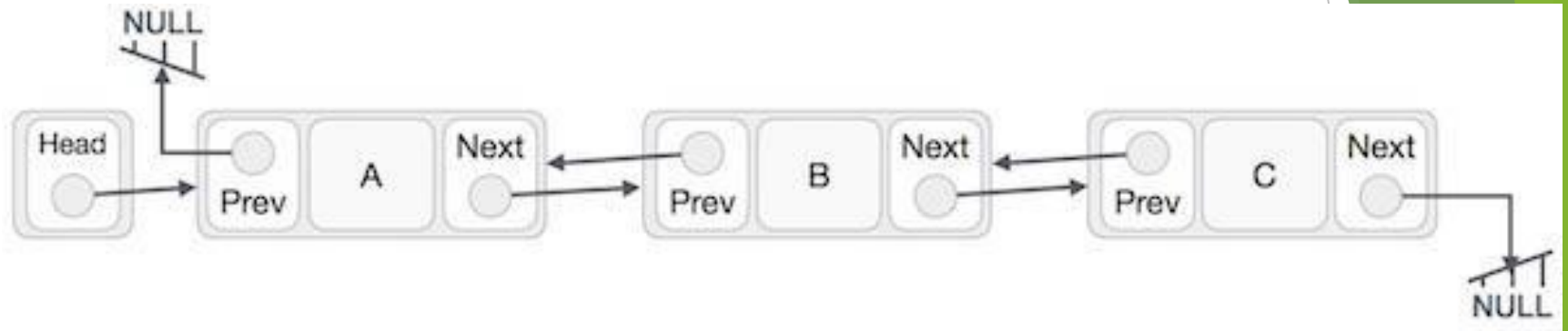
Sort Elements of a Linked List

- ▶ Make the head as the current node and create another node index for later use.
- ▶ If head is null, return.
- ▶ Else, run a loop till the last node (i.e. NULL).
- ▶ In each iteration, follow the following step 5-6.
- ▶ Store the next node of current in index.
- ▶ Check if the data of the current node is greater than the next node. If it is greater, swap current and index.

Doubly Linked Lists

- ▶ Doubly Linked List is a variation of Linked list in which navigation is possible in both ways, forward as well as backward easily as compared to Single Linked List.
- ▶ **Link** – Each link of a linked list can store a data called an element.
- ▶ **Next** – Each link of a linked list contains a link to the next link called Next.
- ▶ **Prev** – Each link of a linked list contains a link to the previous link called Prev.
- ▶ **Linked List** – A Linked List contains the connection link to the first link called First and to the last link called Last.

Doubly Linked List Representation



- ▶ Doubly Linked List contains a link element called first and last.
- ▶ Each link carries a data field(s) and a link field called next.
- ▶ Each link is linked with its next link using its next link.
- ▶ Each link is linked with its previous link using its previous link.
- ▶ The last link carries a link as null to mark the end of the list.

Basic Operations

- ▶ **Insertion** – Adds an element at the beginning of the list.
- ▶ **Insert Last** – Adds an element at the end of the list.
- ▶ **Insert After** – Adds an element after an item of the list.
- ▶ **Deletion** – Deletes an element at the beginning of the list.
- ▶ **Delete Last** – Deletes an element from the end of the list.
- ▶ **Delete** – Deletes an element from the list using the key.
- ▶ **Display forward** – Displays the complete list in a forward manner.
- ▶ **Display backward** – Displays the complete list in a backward manner.

Doubly Linked List - Insertion at the Beginning

- ▶ 1.START
- ▶ 2. Create a new node with three variables: prev, data, next.
- ▶ 3. Store the new data in the data variable
- ▶ 4. If the list is empty, make the new node as head.
- ▶ 5. Otherwise, link the address of the existing first node to the next variable of the new node, and assign null to the prev variable.
- ▶ 6. Point the head to the new node.
- ▶ 7. END

Doubly Linked List - Insertion at the End

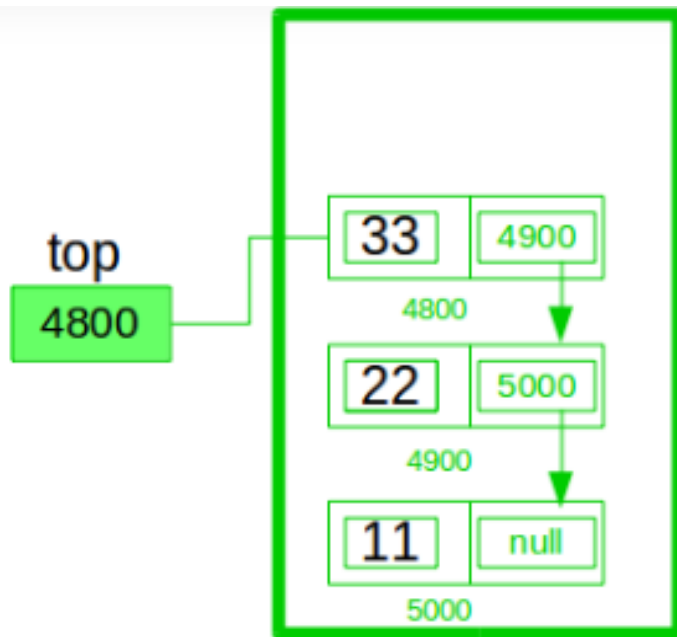
- ▶ 1.START
- ▶ 2. If the list is empty, add the node to the list and point the head to it.
- ▶ 3. If the list is not empty, find the last node of the list.
- ▶ 4. Create a link between the last node in the list and the new node.
- ▶ 5. The new node will point to NULL as it is the new last node.
- ▶ 6. END

Doubly Linked List - Deletion at the Beginning

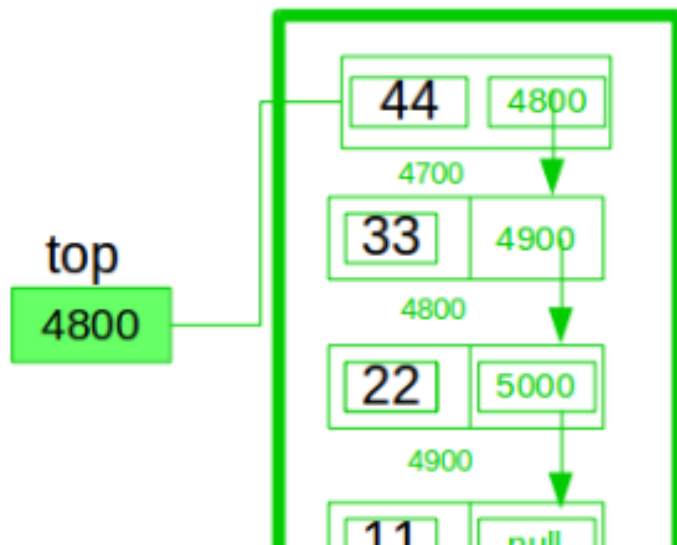
- ▶ 1.START
- ▶ 2. Check the status of the doubly linked list
- ▶ 3. If the list is empty, deletion is not possible 4. If the list is not empty, the head pointer is shifted to the next node.
- ▶ 5. END

Implement a stack using singly linked list

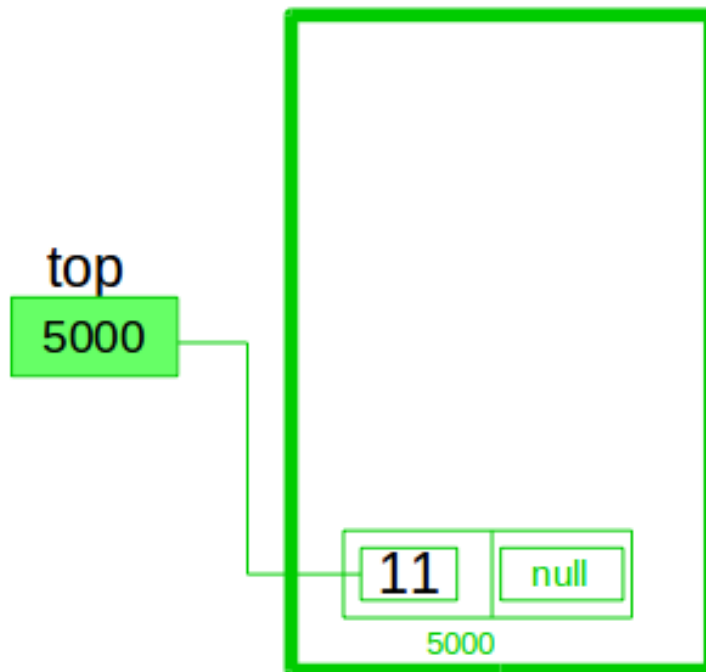
- ▶ To implement a stack using the singly linked list concept, all the singly linked list operations should be performed based on Stack operations LIF.
- ▶ Follow a simple rule in the implementation of a stack which is last in first out and all the operations can be performed with the help of a top variable O(last in first out)



Initial Stack Having Three element
And top have address 4800



First create a temp node and assign 44
Into data field and top in link field
And in last assign temp assign into top



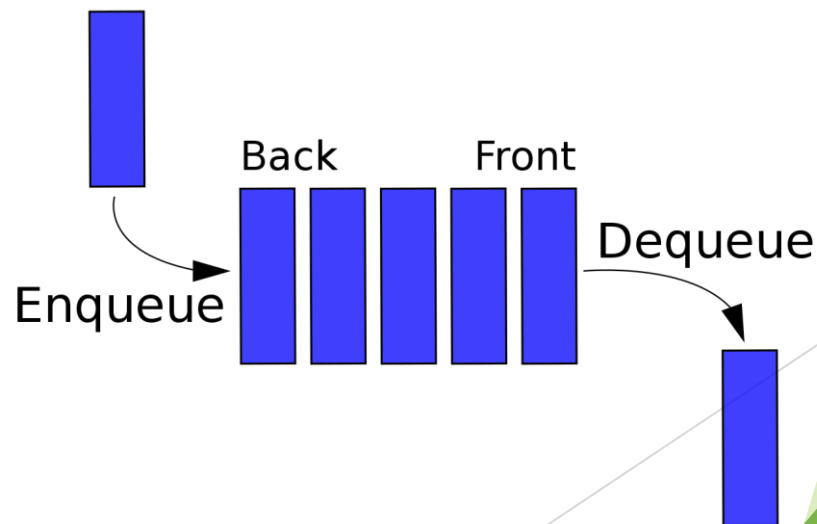
```
Pop three element  
temp = top;  
top = top->link;  
temp->link = NULL;  
free(temp);
```

In the stack Implementation, a stack contains a top pointer. which is the head of the stack where pushing and popping items happens at the head of the list.

The first node has a null in the link field and second node-link has the first node address in the link field and so on and the last node address is in the “top” pointer.

Queue - Linked List Implementation

- ▶ We maintain two pointers, front, and rear. The front points to the first item of the queue and rear points to the last item.
- ▶ enQueue(): This operation adds a new node after the rear and moves the rear to the next node.
- ▶ deQueue(): This operation removes the front node and moves the front to the next node.



Create a class QNode with data members integer data and QNode* next

- A parameterized constructor that takes an integer x value as a parameter and sets data equal to x and next as NULL

2. Create a class Queue with data members QNode front and rear

3. Enqueue Operation with parameter x:

- Initialize QNode* temp with data = x
- If the rear is set to NULL then set the front and rear to temp and return(Base Case)
- Else set rear next to temp and then move rear to temp

4. Dequeue Operation:

- If the front is set to NULL return(Base Case)
- Initialize QNode temp with front and set front to its next
- If the front is equal to NULL then set the rear to NULL
- Delete temp from the memory